



Manual de despliegue

Cómo poner el banco en internet, de cero a HTTPS, en un servidor de 6 dólares por mes.

Octubre de 2026 · orbital.net.ar

Qué vas a montar

Cinco contenedores en un solo servidor. Sólo nginx está expuesto a internet: el resto vive en una red interna y no se puede alcanzar desde afuera.

```
internet → :443 nginx
├── app.orbital.net.ar  el home banking
│   ├── /              → portal (React compilado)
│   ├── /api /auth     → backend (la API)
│   └── /webhooks/     → backend (avisos de Clerk)
└── orbital.net.ar     la landing
```

backend → proveedores (red interna, no sale a internet)
backend → Supabase, Banco Central, DolarAPI, Gemini

La base de datos no se instala. Sigue siendo la de Supabase, que ya está andando. El servidor sólo corre la aplicación.

Lo que necesitás antes de empezar

- Una cuenta en DigitalOcean con una tarjeta cargada.
- El dominio `orbital.net.ar` y acceso a su panel de DNS.
- Las claves: la `DATABASE_URL` de Supabase, las de Clerk y la de Gemini.
- Una clave SSH. Si no tenés, se crea con un comando (paso 1).

Tiempo total: alrededor de una hora, más lo que tarde el DNS en propagar.

Paso a paso

1 Crear la clave SSH (si no tenés)

```
ssh-keygen -t ed25519 -C "maxi"
cat ~/.ssh/id_ed25519.pub
```

Lo que imprime el segundo comando es lo que vas a pegar en DigitalOcean.

2 Crear el droplet

Sistema	Ubuntu 24.04 (LTS) x64
Tipo	Basic · Regular (SSD)
Plan	US\$ 6/mes — 1 GB RAM, 1 CPU, 25 GB SSD
Región	New York, la de mejor latencia desde Argentina
Autenticación	SSH Key → <i>New SSH Key</i> y pegás la del paso 1

Anotá la IP que te asigna: se usa en todos los pasos que siguen.

3 Delegar el dominio

En **nic.ar**, en el dominio → *Delegaciones*, cargá los nameservers de DigitalOcean:

```
ns1.digitalocean.com
ns2.digitalocean.com
ns3.digitalocean.com
```

Después, en DigitalOcean → *Networking* → *Domains*, agregás `orbital.net.ar` y creás tres registros **A** apuntando a la IP del droplet:

Hostname	Para qué
@	la landing
www	redirige a la landing
app	el home banking

Comprabá que resuelva antes de seguir con los certificados:

```
dig +short app.orbital.net.ar
```

Si no devuelve la IP del droplet, **esperá**: la delegación de NIC.ar puede tardar horas. Si seguís igual, el certificado va a fallar, y Let's Encrypt sólo permite 5 intentos fallidos por hora.

4 Entrar y dejarlo seguro

```
ssh root@LA_IP

adduser maxi && usermod -aG sudo maxi
rsync --archive --chown=maxi:maxi ~/.ssh /home/maxi

ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw --force enable
apt update && apt install -y unattended-upgrades
```

Queda abierto sólo SSH, HTTP y HTTPS, y los parches de seguridad se instalan solos.

5 Swap y Docker

```
fallocate -l 1G /swapfile && chmod 600 /swapfile
mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab

curl -fsSL https://get.docker.com | sh
usermod -aG docker maxi
exit
```

Con 1 GB la swap es una red de seguridad, no una necesidad. Después de agregar el usuario al grupo docker hay que volver a entrar: `ssh maxi@LA_IP`

6 Traer el código

```
sudo mkdir -p /opt/orbital && sudo chown maxi:maxi /opt/orbital
cd /opt/orbital

for r in backend frontend proveedores landing infra; do
  git clone https://github.com/maxibertaina03/banco-$r.git
done
```

7 Cargar las claves

```
cd /opt/orbital/banco-infra
cp .env.example .env
cp env/backend.env.example env/backend.env
cp env/proveedores.env.example env/proveedores.env

nano .env          # dominios, mail y clave publicable de Clerk
nano env/backend.env # base de datos, Clerk, Gemini
nano env/proveedores.env

chmod 600 .env env/*.env
```

La base de datos. En DATABASE_URL usá la dirección del *Session Pooler* de Supabase (Connect → Session pooler), no la conexión directa: el servidor sale a internet por IPv4 y la conexión directa puede no estar disponible por ahí.

Estos archivos tienen contraseñas. Nunca se suben a GitHub: viven solamente en el servidor.

8 Levantar todo

```
docker compose up -d --build
docker compose ps
```

La primera vez tarda unos minutos: tiene que compilar el portal y la landing.

Los servicios con chequeo de salud deben decir healthy. Para probar la API sin tener todavía el dominio:

```
docker compose exec -T backend node -e
"fetch('http://127.0.0.1:3001/api/health').then(r=>r.json()).then(j=>console.log(j))"
```

9 Sacar los certificados de HTTPS

```
./scripts/certificados.sh --prueba # ensayo, no gasta intentos
./scripts/certificados.sh          # los de verdad
```

El script pide el certificado, activa HTTPS y recarga nginx. Las renovaciones después se hacen solas: hay un contenedor que se encarga cada doce horas.

10 Avisarle a Clerk

- Agregar `https://app.orbital.net.ar` a los dominios permitidos.
- Apuntar el webhook a `https://app.orbital.net.ar/webhooks/clerk`.

Con las claves de desarrollo el portal funciona igual, con un cartelito que dice "Development mode". Pasar a producción se puede hacer después.

11 Comprobar que quedó bien

```
curl -s https://app.orbital.net.ar/api/health
curl -I https://orbital.net.ar
docker compose run --rm --entrypoint certbot certbot renew --dry-run
```

El primero tiene que responder `"ok":true` y `"db":"connected"`. El tercero confirma que el certificado se va a renovar solo dentro de 60 días.

La prueba final: reiniciá el servidor con `sudo reboot` y esperá un minuto. Si todo vuelve solo, el banco quedó realmente "en la nube". Después entrá desde el celular con datos móviles, que valida el dominio, el certificado y el diseño en pantalla chica de una sola vez.

El día a día

Todo se hace desde `/opt/orbital/banco-infra`.

Comando	Qué hace
<code>./scripts/deploy.sh</code>	Trae los cambios de GitHub y actualiza todo
<code>./scripts/deploy.sh backend</code>	Actualiza un solo servicio
<code>docker compose ps</code>	Ver qué está corriendo y cómo está de salud
<code>docker compose logs -f backend</code>	Mirar la API en vivo
<code>docker compose restart backend</code>	Reiniciar sin cortar operaciones en curso
<code>free -h / df -h /</code>	Memoria y espacio en disco

Las migraciones de la base no las corre el servidor. Se siguen aplicando desde tu computadora contra Supabase, como venías haciendo. Es a propósito: así es muy difícil romper la base sin querer.

Si algo falla

Qué ves	Qué mirar
502 Bad Gateway	Algún contenedor se cayó. <code>docker compose ps</code> y después <code>docker compose logs backend</code> .
El portal carga en blanco	Casi siempre es el navegador con el archivo viejo en caché. Recargá con <code>Ctrl + Shift + R</code> .
No aparece el login de Clerk	El dominio no está permitido en Clerk, o la clave con la que se compiló está mal. Ojo: cambiar esa clave obliga a reconstruir, no alcanza con reiniciar.
<code>"db":"disconnected"</code>	La <code>DATABASE_URL</code> , o Supabase pausó el proyecto por inactividad.
El certificado no sale	Casi siempre el DNS todavía no resolvía. Verificá con <code>dig +short app.orbital.net.ar</code> y reintentá con <code>--prueba</code> para no gastar intentos.
No queda espacio en disco	<code>docker system prune -af. Sin --volumes</code> : esa bandera borraría los certificados.

Cuánto cuesta

Concepto	Precio
Droplet de DigitalOcean (1 GB)	US\$ 6 por mes
Dominio orbital.net.ar	\$ 8.500 por año
Supabase (base de datos)	gratis
Clerk (autenticación)	gratis
Certificados Let's Encrypt	gratis
Total	US\$ 6 por mes + el dominio una vez al año

La versión larga de este manual, con el detalle técnico de cada archivo, está en el README.md del repositorio `banco-infra`.